

Day 15 Evidence Workbooklet

Lists, Indexing, List State, and Reading List Evidence

Engineering practice to stored sequence to index to mutation to looped list evidence

Student copy. Guided workbooklet, not the mastery quiz. Print format: duplex, left-stapled packet.

Name		Section		Date	
Score	____/46		Mastery check	secure / developing / needs support	

Concrete engineering situation

By the end of Week 3, the checkout desk needs to store repeated evidence rather than only print it. Day 15 introduces lists as a way to keep a sequence of values, read values by index, update list state, and loop over stored evidence.

Engineering task	Read and update a stored sequence of checkout evidence without confusing positions, values, and list state.
Computational move	Use list positions and looped list reads to inspect repeated evidence that is stored together.
Python focus	list literals, index positions, len, append, item update, loop over list, loop over range(len(...)), and list state trace.
Evidence to keep	list length, index, value at index, state before mutation, state after mutation, appended item, and looped value trace.

Day 15 working rules

1. A list stores values in order.
2. Python list indexing starts at 0.
3. len(list_name) counts items, not the last index.
4. append changes the list state by adding an item at the end.
5. When a loop reads a list, keep index evidence and value evidence separate.

Point map **Manage your evidence**

blueprint

Points	Section	Evidence focus
1	Read list length, positions, and values	recognize, index
2	Separate index evidence from value evidence	trace, state
3	Trace append and item update	mutation, state
4	Loop over list values to count evidence	loop, accumulate
5	Debug an indexing mistake	debug, boundary
6	Transfer and support route	transfer, reflect

1 Read list length, positions, and values

[recognize](#)[index](#)

A list lets the checkout desk store repeated values together. Index positions start at 0.

```
1 charges = [82, 77, 91, 80]
2
3 print(len(charges))
4 print(charges[0])
5 print(charges[3])
```

Pts.	Prompt	My evidence / response
1	How many items are in charges?	
1	What value is at index 0?	
1	What value is at index 3?	
1	Why would charges[4] be an error?	

2 Separate index evidence from value evidence

trace state

When reading a list, the index and the stored value are different pieces of evidence.

```
1 charges = [82, 77, 91, 80]
2 for index in range(len(charges)):
3     print(index, charges[index])
```

Pts.	Prompt	My evidence / response
2	Write the first two printed lines.	
2	Write the last two printed lines.	
2	What values does range(len(charges)) produce?	
2	What is the difference between index and charges[index]?	
2	Why is len(charges) not used as a final index?	

3 Trace append and item update

mutation state

List operations can change stored state. Trace before and after each change.

```
1 routes = ["release", "charge"]
2 routes.append("repair")
3 routes[1] = "hold"
4 print(routes)
```

Pts.	Prompt	My evidence / response
2	What is the list state before append?	
2	What is the list state after append?	
2	Which item changes after routes[1] = "hold"?	
2	What final list prints?	
2	Explain why append and index assignment are both mutations.	

4 Loop over list values to count evidence

loop accumulate

A loop can read stored list values and update a counter from the evidence.

```
1 charges = [82, 77, 91, 80]
2 ready_count = 0
3
4 for charge in charges:
5     if charge >= 80:
6         ready_count = ready_count + 1
7
8 print(ready_count)
```

Pts.	Prompt	My evidence / response
2	Which values does charge take in order?	
2	For each value, decide whether charge >= 80 is true.	
2	Trace ready_count after each list item.	
2	What final count prints?	
2	Why is this loop easier than writing four separate if statements?	

5 Debug an indexing mistake

debug

boundary

The intended task is to print every charge value. This loop has an indexing boundary bug.

```
1 charges = [82, 77, 91, 80]
2 for index in range(1, len(charges)):
3     print(charges[index])
```

Pts.	Prompt	My evidence / response
2	Which charge value is skipped?	
2	Why does range(1, len(charges)) skip it?	
2	Write the corrected range call if you want to use indexes.	
2	Write a version that loops directly over charge values instead.	
2	Name a test that would catch the skipped-first-item bug.	

Common mistake to avoid

The first list index is 0. Starting an index loop at 1 skips the first item.

6 Transfer and support route

transfer reflect

Close Week 3 by connecting lists back to Exam 2 readiness. Lists make repeated evidence visible and inspectable.

Pts.	Prompt	My evidence / response
4	Write a short note explaining how a list plus a loop changes repeated checkout evidence from scattered values into a stored sequence.	
2	Circle your support route: index / value / length / mutation / looped count / boundary. Name one next action.	
2	Name one Week 3 skill that should appear on the later bridge Exam 2.	

Support route
Week 4 can now combine list mutation, list-plus-loop processing, and Exam 2 review. Keep index/value/state evidence visible; do not jump straight to final answers.

Copyright © 2026 Lance L.A. White. All rights reserved.